

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки
09.03.01 Информатика и вычислительная техника(по отраслям)
направленность (профиль)
«Разработка программного обеспечения и обработка больших данных»

Курсовая работа

**по дисциплине «Прикладные информационны
технологии»**

«Разработка и контейнеризация веб-приложения для управления
личной библиотекой с использованием Docker и автоматизация
выпуска сертификатов Let's Encrypt»

Обучающегося 3 курса
очной формы обучения
Смородина А.А.

Руководитель курсовой работы:
старший преподаватель кафедры ИТЭО
Аксютин Павел Александрович

Санкт-Петербург

2026

Оглавление

Введение	3
1.1 Актуальность темы исследования	3
1.2 Цель и задачи исследования	3
1.3 Задачи	3
1.4 Объект и предмет исследования	3
1.5 Методы исследования	4
1.6 Практическая значимость	4
2. Теоретическая часть.....	5
2.1. Контейнеризация и платформа Docker	5
2.2. Оркестрация с помощью Docker Compose	5
2.3. Веб-сервер NGINX как Reverse-Proxy.....	5
2.4. Безопасность HTTPS и автоматизация Let's Encrypt	6
2.5. Выбор технологического стека	6
3. Программная реализация TaskTracker Pro	7
3.1. Архитектура системы.....	7
3.2. Серверная часть (Backend) и База данных.....	7
3.3. Клиентский интерфейс (Frontend)	7
3.4. Контейнеризация и Оркестрация (Docker Compose)	7
3.5. Настройка Nginx и Автоматизация HTTPS.....	7
4. Результат работы.....	8
4.2.Интерфейс приложения.....	8
4.2. Документация по запуску	8
Приложение А. Исходный код.....	9
А.1. Файл docker-compose.yml.....	9
А.2. Файл Dockerfile.....	10
А.3. Файл app.py	10

Введение

1.1 Актуальность темы исследования

В условиях цифровизации личных коллекций, управление домашней библиотекой требует инструментов, обеспечивающих доступность и структурированность данных. Приложение «BookShelf Manager», направленное на эффективный учет книг, требует современных подходов к инфраструктуре. Использование контейнеризации и автоматизация SSLсертификатов Let's Encrypt позволяют создать защищенное и надежное веб-решение.

1.2 Цель и задачи исследования

Цель — разработать веб-приложение «BookShelf Manager», реализовать его контейнеризацию, настроить NGINX и автоматизировать процесс выпуска SSL-сертификатов.

1.3 Задачи

- Разработать API на Python (Flask) для учета книг в библиотеке.
- Реализовать интерфейс для добавления, отображения и удаления книг.
- Подготовить конфигурации Dockerfile и docker-compose.yml.
- Настроить NGINX в качестве reverse-proxy.
- Автоматизировать HTTPS через Let's Encrypt.
- Провести тестирование системы.

1.4 Объект и предмет исследования

1. Объект исследования: процесс разработки и развертывания веб-приложений.
2. Предмет исследования: технологии контейнеризации (Docker, Docker Compose), reverse-proxy сервер NGINX и методы автоматизации HTTPS (Let's Encrypt, ACME Companion).

1.5 Методы исследования

- 1. Метод разработки RESTful API для взаимодействия компонентов системы.
- 2. Метод контейнеризации и оркестрации сервисов с использованием Docker Compose.
- 3. Метод настройки reverse-проxy для обеспечения безопасности и управления трафиком.
- 4. Метод функционального тестирования клиент-серверного взаимодействия.
-

1.6 Практическая значимость

Практическая значимость работы заключается в создании функционального веб-приложения «BookShelf Manager», готового к эксплуатации в защищенной среде HTTPS. Разработанная архитектура на основе Docker Compose и NGINX-Проxy может выступать шаблоном для быстрого развертывания аналогичных веб-сервисов.

2. Теоретическая часть

2.1. Контейнеризация и платформа Docker

Контейнеризация представляет собой технологию изоляции программного обеспечения, позволяющую упаковать приложение вместе со всеми его зависимостями, библиотеками и конфигурационными файлами в единый стандартизированный образ. В отличие от использования полнофункциональных виртуальных машин, контейнеры работают непосредственно на ядре хостовой операционной системы. Такой подход обеспечивает минимальные затраты ресурсов, высокую скорость развертывания и предсказуемость поведения приложения в любых окружениях.

Docker является ключевой экосистемой для управления контейнерами. Основное преимущество использования Docker заключается в устранении расхождений между средой разработки и продуктовым сервером, что позволяет гарантировать идентичную работу приложения независимо от инфраструктуры.

2.2. Оркестрация с помощью Docker Compose

Реальные веб-приложения зачастую представляют собой сложную систему, состоящую из множества взаимодействующих сервисов: веб-сервера, API-бэкенда и

систем хранения данных. Управление такими компонентами вручную является трудоемким и подверженным ошибкам процессом.

Docker Compose — это инструмент для автоматизации развертывания многоконтейнерных приложений. Инфраструктура описывается с помощью декларативного YAML-файла (`docker-compose.yml`), в котором определяются параметры сети, тома для персистентного хранения данных и настройки образов. Это обеспечивает возможность запуска всей экосистемы проекта одной командой и упрощает масштабирование.

2.3. Веб-сервер NGINX как Reverse-Proxy

В данной работе NGINX выполняет роль обратного прокси-сервера (`reverse-proxy`). Основные задачи включают:

1. Обработка статического контента: эффективная отдача HTML, CSS и JS-файлов, что снижает нагрузку на бэкенд.
2. Маршрутизация запросов: NGINX принимает все входящие соединения из внешней сети и проксирует их к внутренним контейнерам приложения. Это обеспечивает дополнительный уровень безопасности, скрывая детали

внутренней архитектуры от внешних пользователей.

2.4. Безопасность HTTPS и автоматизация Let's Encrypt

Шифрование трафика посредством протокола HTTPS является обязательным требованием современного веба. Для автоматизации процесса получения и продления SSL-сертификатов в проекте используются центр сертификации Let's Encrypt и утилита ACME Companion (или Certbot). Данное решение позволяет поддерживать валидность сертификатов в автоматическом режиме, минимизируя риск простоев из-за истечения срока действия ключей шифрования.

2.5. Выбор технологического стека

При проектировании «BookShelf Manager» были выбраны:

Бэкенд: Flask (Python) из-за его легковесности и гибкости. База данных:

SQLite, так как она не требует настройки отдельного выделенного контейнера базы данных.

3. Программная реализация BookShelf Manager

3.1. Архитектура системы

Приложение реализовано на основе клиент-серверной архитектуры и развернуто в виде микросервисной системы, управляемой Docker Compose. Общая схема включает сервис nginx-проxy и bookshelf_app.

3.2. Серверная часть (Backend) и База данных

Бэкенд написан на Python с использованием Flask. Он обрабатывает запросы на добавление и удаление записей о книгах. Для обеспечения персистентности данных (сохранения записей после перезапуска контейнера) в конфигурации используется том (volume).

3.3. Клиентский интерфейс (Frontend)

Пользовательский интерфейс интегрирован в бэкенд и реализован с использованием HTML и фреймворка Bootstrap. Пользователь может просматривать список книг, добавлять новые записи и удалять их.

3.4. Контейнеризация и Оркестрация (Docker Compose)

Контейнеризация осуществлена с помощью Dockerfile, базирующегося на образе python:3.9-slim. Управление жизненным циклом всей системы описано в декларативном YAML-файле (docker-compose.yml). Он определяет два сервиса: reverse-проxy (nginx-проxy) и веб-приложение (app). Контейнер приложения настроен на автоматическую интеграцию с прокси-сервером через переменные окружения VIRTUAL_HOST и VIRTUAL_PORT. Для надежности используется volume для файла базы данных.

3.5. Настройка Nginx и Автоматизация HTTPS

Для обеспечения безопасности соединения в конфигурации Nginx реализован Reverse-Proxy. Автоматическое получение SSL-сертификатов для домена assmi.duckdns.org реализуется через companion-контейнер.

4. Результат работы

Разработанное приложение успешно функционирует в контейнеризированной среде, обеспечивая безопасный доступ по HTTPS и удобный интерфейс для управления личной библиотекой.

4.2. Интерфейс приложения

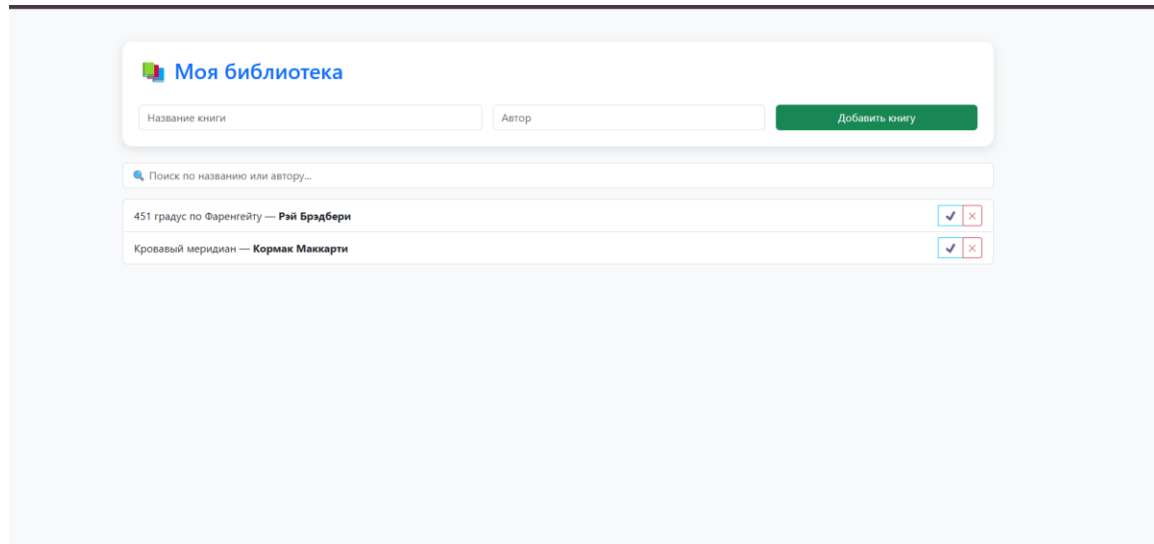


Рисунок 4.2.1 – тестирование интерфейса

4.2. Документация по запуску

1. Настройте DNS-запись домена (A-запись `assmi.duckdns.org` на IP сервера).
2. Убедитесь, что на сервере установлены Docker и Docker Compose.
3. Скопируйте файлы проекта в директорию сервера.
4. Выполните команду `"docker compose up -d --build"`.
5. Приложение будет доступно по адресу `http://assmi.duckdns.org`.

Приложение А. Исходный код

А.1. Файл docker-compose.yml

```
services:
  nginx-proxy:
    image: nginxproxy/nginx-proxy
    container_name: nginx-proxy
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /var/run/docker.sock:/tmp/docker.sock:ro
      - certs:/etc/nginx/certs:ro
      - vhost.d:/etc/nginx/vhost.d
      - html:/usr/share/nginx/html
    networks:
      - webnet

  acme-companion:
    image: nginxproxy/acme-companion
    container_name: acme-companion
    volumes_from:
      - nginx-proxy
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - acme:/etc/acme.sh
    networks:
      - webnet

  app:
    build: .
    container_name: bookshelf_app
    ports:
      - "5000:5000"
    environment:
      - VIRTUAL_HOST=assmi.duckdns.org
      - VIRTUAL_PORT=5000
      - LETSENCRYPT_HOST=assmi.duckdns.org
      - LETSENCRYPT_EMAIL=ls481693@gmail.com
    volumes:
      - ./data:/app/data
    networks:
      - webnet

networks:
  webnet:

volumes:
  certs:
  vhost.d:
  html:
  acme:
```

A.2. Файл Dockerfile

```
FROM python:3.9-slim
WORKDIR /app
RUN pip install flask
COPY . .
CMD ["python", "app.py"]
```

A.3. Файл app.py

```
from flask import Flask, request, render_template_string
import sqlite3
import os

app = Flask(__name__)
DB_PATH = 'data/library.db'

def init_db():
    if not os.path.exists('data'): os.makedirs('data')
    conn = sqlite3.connect(DB_PATH)
    # Добавили поле 'status' (0 - не прочитано, 1 - прочитано)
    conn.execute('''CREATE TABLE IF NOT EXISTS books
                    (id INTEGER PRIMARY KEY, title TEXT, author TEXT,
status INTEGER DEFAULT 0)''')
    conn.commit()
    conn.close()

@app.route('/', methods=['GET', 'POST'])
def index():
    conn = sqlite3.connect(DB_PATH)
    if request.method == 'POST':
        if 'add' in request.form:
            title, author = request.form['title'],
request.form['author']
            conn.execute('INSERT INTO books (title, author) VALUES (?,
?)', (title, author))
        elif 'del' in request.form:
            conn.execute('DELETE FROM books WHERE id = ?',
(request.form['id'],))
        elif 'toggle' in request.form:
            conn.execute('UPDATE books SET status = 1 - status WHERE id
= ?', (request.form['id'],))
            conn.commit()

    search = request.args.get('q', '')
    query = 'SELECT * FROM books WHERE title LIKE ? OR author LIKE ?'
    books = conn.execute(query, (f'%{search}%',
f'%{search}%')).fetchall()
    conn.close()

    html = """
    <!DOCTYPE html>
    <html lang="ru"><head>
    <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.m
in.css" rel="stylesheet">
    <style>
        body { background: #f8f9fa; }
        .card { border-radius: 15px; border: none; box-shadow: 0 4px
15px rgba(0,0,0,0.1); }
        .read { text-decoration: line-through; color: #6c757d; }
    </style></head><body>
    <div class="container mt-5">

```

```

        <div class="card p-4 mb-4">
            <h2 class="text-primary">📖 Моя библиотека</h2>
            <form method="POST" class="row g-3 mt-2">
                <div class="col-md-5"><input name="title"
placeholder="Название книги" class="form-control" required></div>
                <div class="col-md-4"><input name="author"
placeholder="Автор" class="form-control" required></div>
                <div class="col-md-3"><button name="add" class="btn
btn-success w-100">Добавить книгу</button></div>
            </form>
        </div>

        <form class="mb-3"><input name="q" class="form-control"
placeholder="🔍 Поиск по названию или автору..."></form>

        <ul class="list-group">
            {% for b in books %}
                <li class="list-group-item d-flex justify-content-between
align-items-center">
                    <span class="{{ 'read' if b[3] == 1 }}">{{ b[1] }} -
<b>{{ b[2] }}</b></span>
                    <form method="POST" class="btn-group">
                        <input type="hidden" name="id" value="{{ b[0] }}">
                        <button name="toggle" class="btn btn-sm btn-
outline-info">✔</button>
                        <button name="del" class="btn btn-sm btn-outline-
danger">✕</button>
                    </form>
                </li>
            {% endfor %}
        </ul>
    </div></body></html>
    """
    return render_template_string(html, books=books)

if __name__ == '__main__':
    init_db()
    app.run(host='0.0.0.0', port=5000)

```